

How Computers Work · Section 6

Processes, Threads, and Memory

By TOUCH DEN · November 11, 2026

Contents

What You Will Learn 1

Before You Start 1

1. What a Process Is 1

2. How a Process Is Laid Out in Memory 2

3. The Stack and the Heap 2

4. Threads 3

5. Virtual Memory and Isolation 4

6. How the OS Keeps Processes Apart 4

7. Looking at Running Processes 5

Common Mistakes 6

Summary 6

Check Your Understanding 6

Practice 7

What You Will Learn

- What a process is, and how it differs from a program.
- How a process is laid out in memory.
- The stack and the heap.
- Threads, and how they relate to processes.
- Virtual memory, and how it isolates processes.
- How the OS keeps processes apart.
- How to look at the running processes on a real machine.

Before You Start

Read lessons 4 and 5. You know the CPU runs instructions and the OS manages programs. This lesson zooms into a single running program: a **process**. This is one of the most important lessons in the course, because the memory layout here is the ground the Binary Exploitation (18) and Reverse Engineering (17) courses stand on. The console example uses the `ps` command; the layout shown is the real format, with illustrative values.

---

1. What a Process Is

A **program** is a file sitting on disk: a list of instructions, not doing anything. A **process** is a program that is *running*: loaded into memory, with the CPU executing its instructions. The difference is like a recipe versus actually cooking the dish. One recipe (program) can be cooked many times at once (several processes), which is why you can open two windows of the same app.

When you start a program, the OS creates a process for it:

- It loads the program's instructions from disk into RAM.
- It gives the process its own chunk of memory to work in.
- It assigns a unique number, the **PID** (Process ID), to track it.
- It starts the CPU executing the process's instructions (sharing the CPU with all the other processes, lesson 5).

Every process also has a **parent**: the process that started it, identified by its **PPID** (Parent Process ID). When you launch an app, the process that launched it is its parent, and processes form a family tree back to the first process the OS starts at boot (lesson 9). This parent–child

structure matters in security: investigators and defenders look at *what started what*. A word processor spawning a command shell, or a browser launching an unknown program, is the kind of unusual parent–child relationship that reveals an attack.

2. How a Process Is Laid Out in Memory

When a process runs, the OS gives it a block of memory, and that memory is organized into regions, each with a job. Knowing this map is essential for everything that follows. From the bottom (low addresses) to the top (high addresses), a process's memory holds:

Region	What it holds
Code (text)	The program's instructions themselves
Data	Global variables and constants the program set up
Heap	Memory the program requests while running, grows **upward**
Stack	Temporary data for functions, grows **downward**

Picture it as a tall strip of memory. The program's fixed parts, its instructions (code) and its set-up data, sit at the bottom. Above them, two regions grow *toward each other*: the **heap** grows up from below, and the **stack** grows down from the top. Between them is free space they can grow into.

Why does a security course care about this map? Because attacks live in it. The **code** region is what a reverse engineer reads. The **stack** is where classic buffer-overflow attacks happen. The **heap** is where a different family of attacks happens. Keeping or breaking the boundaries between these regions is the entire game of memory-corruption exploitation (course 18). For now, just fix the picture: a running process is a strip of memory with instructions at the bottom and two regions, stack and heap, growing toward each other.

3. The Stack and the Heap

Two of those regions, the stack and the heap, are where a program keeps the data it works with while running. They behave very differently, and the difference matters.

The stack is for short-lived, orderly data, especially the data of **functions** (a function is a named block of instructions a program can call). Every time a function is called, a chunk called a **stack frame** is added to the top of the stack, holding that function's local variables and, crucially, the **return address**: where to continue when the function finishes. When the function returns, its frame is removed. The stack grows and shrinks neatly, last-in-first-out, like a stack of plates.

That return address on the stack is one of the most important facts in all of security. A **buffer overflow** attack works by writing *past* the end of a buffer on the stack until it overwrites the return address, so that when the function finishes, the CPU jumps to an address the *attacker* chose, running the attacker's instructions. Everything in this topic, functions, frames, the return address, is the setup for understanding that attack in lesson 11 and in Binary Exploitation.

The heap is for longer-lived data whose size is not known in advance, memory the program explicitly requests while running ("give me room for this image I just loaded"). The heap is more flexible but less orderly than the stack, and the program is responsible for giving memory back when done. Mistakes here (using memory after giving it back, or forgetting to give it back) cause another whole class of bugs and exploits (**use-after-free**, memory leaks), studied later. The short version: **stack = orderly, automatic, function data; heap = flexible, manual, longer-lived data.**

4. Threads

A process can do more than one thing at a time by using **threads**. A thread is a single stream of execution within a process, a thread of instructions being run. A process always has at least one thread (the program running), and it can create more to do several things at once: one thread handling the screen while another downloads a file, for example.

The key distinction:

- **Processes are isolated from each other.** Each has its own separate memory (topics 5–6). One process cannot see into another's memory.
- **Threads within one process share that process's memory.** They are all inside the same memory block, working on the same data.

That sharing makes threads fast and convenient, but it is also a source of difficulty and bugs. Because threads share memory and run at unpredictable, interleaved times (lesson 4's context switching), two threads can collide, both changing the same data at once, leading to **race conditions**: bugs where the outcome depends on the exact timing of threads. Race conditions are not just reliability bugs; they are a real security vulnerability class (for example, checking a permission in one thread while another changes the thing being checked). You will meet them in Web Application Security and exploitation. For now: a thread is a stream of execution inside a process; threads share memory, which is powerful and dangerous.

5. Virtual Memory and Isolation

Here is a question that reveals something deep: if every process is just put into RAM, what stops one process from reading or trashing another's memory, or the kernel's? The answer is **virtual memory**, one of the most important ideas in how computers work.

The trick: **each process is given its own private view of memory, as if it were the only program on the machine.** A process sees a clean, continuous range of memory addresses starting from zero, called its **virtual address space**, and it believes it owns all of it. But those addresses are not real physical locations. Behind the scenes, the CPU and OS translate each process's virtual addresses into actual physical RAM locations, and they make sure each process's virtual memory maps to a *different*, separate part of physical RAM.

The consequences are profound:

- **Isolation.** Because each process has its own private address space, one process literally cannot name, let alone touch, another process's memory. Address `0x1000` in one process and `0x1000` in another point to completely different physical memory. This is the main thing keeping your banking app's memory safe from a malicious app running beside it.
- **Simplicity for programs.** Every program can be written as if it has the whole machine to itself, and the OS sorts out the real locations.

Virtual memory is why processes are isolated, and that isolation is a core security guarantee, the same theme as lesson 5's user/kernel boundary. When you hear that an attack "broke out of" a process, or read another process's memory, it means they defeated this isolation, which is exactly why doing so is so serious.

6. How the OS Keeps Processes Apart

Virtual memory gives each process its own space; the OS and CPU work together to *enforce* that the walls hold. A few mechanisms, which you only need to recognize:

- **Separate address spaces** (topic 5): each process's memory maps to different physical RAM, so they cannot name each other's data.
- **Permission bits on memory regions.** Each region of a process's memory is marked with what is allowed: readable, writable, executable. For example, the **code** region is executable but not writable (so a program cannot easily rewrite its own instructions), and the **stack** is writable but, on modern systems, *not* executable, a defense (called **NX**, no-execute) specifically designed to stop the classic "run my instructions off the stack" attack. You will meet NX and related defenses by name in Binary Exploitation.

- **The user/kernel boundary** (lesson 5): processes cannot reach the kernel's memory or each other's except through controlled system calls.

The theme across lessons 4, 5, and 6 is now clear: a computer is a set of **boundaries**, process from process, user from kernel, readable from writable from executable, all enforced by the CPU and OS together. Security, at the machine level, is largely the story of keeping those boundaries intact, and attacks are largely the story of breaking them. This lesson has shown you the boundaries around a single process; the exploitation course is about defeating them, and the defense courses about reinforcing them.

7. Looking at Running Processes

All of this is real and visible on any computer. The command that lists running processes is `ps` (on Linux and macOS; Windows has Task Manager and `tasklist`). A typical result looks like this:

```
$ ps -o pid,ppid,%cpu,%mem,comm
  PID  PPID %CPU %MEM COMM
 1521     1   0.3  1.8 WindowServer
 2983  2970   1.2  4.1 Google Chrome
 3001  2983   0.0  2.2 Google Chrome Helper
 4102  3550   0.1  0.4 zsh
```

Read the columns with what you now know:

- **PID**: the process's unique ID.
- **PPID**: its parent's PID, the process that started it. Notice `Google Chrome Helper` (PID 3001) has PPID 2983, which is `Google Chrome`, the browser started its own helper. That parent–child link is topic 1 made visible.
- **%CPU** and **%MEM**: how much processor time and memory the process is using.
- **COMM**: the program's name.

This simple view is the foundation of a lot of defensive work. A SOC analyst or incident responder (courses 25, 27) scans process lists for the abnormal: a process with a strange name, an unusual parent (a document viewer whose child is a command shell), or one eating CPU for no reason. Malware hides among processes, and finding it starts with being able to read a list like this. You will do this hands-on in lesson 12's project, and constantly in the defensive courses.

You now understand a running process inside and out: what it is, how its memory is laid out, the stack and heap, threads, and the isolation that keeps processes apart. Next, we look at where programs and data rest when they are *not* running: files and file systems.

Common Mistakes

1. Confusing a program with a process. A program is a file on disk; a process is that program running in memory. One program can have many processes.

2. Forgetting the return address on the stack. It is the single most targeted value in classic exploitation: overflow a buffer, overwrite it, and redirect the CPU. The whole stack layout matters.

3. Thinking threads are just like separate programs. Threads share their process's memory, which makes them fast but causes race conditions. Processes are isolated; threads are not.

4. Assuming processes can freely read each other's memory. Virtual memory gives each its own private address space; they cannot, unless isolation is defeated, which is a serious attack.

5. Overlooking parent–child relationships. *What started what* is a key signal. An unusual parent (e.g. a document app spawning a shell) is a classic sign of compromise.

Summary

Term	Remember
Program vs process	A file on disk vs that program running in memory
PID / PPID	A process's ID / its parent's ID (what started it)
Memory layout	Code and data at the bottom; heap grows up, stack grows down
Stack	Orderly, automatic function data; holds the **return address** (the exploit target)
Heap	Flexible, manual, longer-lived requested memory
Thread	A stream of execution inside a process; threads **share** memory (→ race conditions)
Virtual memory	Each process gets a private address space → isolation
Keeping apart	Separate address spaces, memory permission bits (NX), the user/kernel boundary

Check Your Understanding

1. What is the difference between a program and a process?
2. What do PID and PPID mean, and why is the parent–child link useful in security?
3. Name the four regions of a process's memory, and say which two grow toward each other.
4. What is stored in a stack frame, and which value in it do buffer-overflow attacks target?

5. How do threads differ from processes in terms of memory, and what bug class does that cause?
6. What is virtual memory, and how does it isolate processes?
7. In a `ps` list, you see a PDF reader as the parent of a command shell. Why might that worry a defender?

Answers

1. A program is a file of instructions on disk, doing nothing; a process is that program running, loaded into memory with the CPU executing it. One program can run as many processes.
2. PID is a process's unique ID; PPID is its parent's PID, the process that started it. The link shows *what started what*, and unusual parent–child relationships reveal attacks.
3. Code (text), data, heap, and stack. The heap (growing up) and the stack (growing down) grow toward each other.
4. A function's local variables and the return address (where to continue after the function finishes). Buffer overflows target the return address, to redirect the CPU.
5. Threads within a process share that process's memory (processes are isolated from each other). Sharing causes race conditions, bugs where the outcome depends on thread timing.
6. Giving each process its own private view of memory (a virtual address space) that the CPU/OS translate to different physical RAM. Because addresses are private and map to separate RAM, one process cannot name or touch another's memory.
7. Because a PDF reader does not normally launch a command shell; that unusual parent–child relationship is a classic sign that the document exploited the reader to run commands.

Practice

1. Explain, with the recipe/cooking analogy, why you can have two windows of the same app open at once.
2. Draw (or describe in words) the memory layout of a process: put code, data, heap, and stack in order, and show which way the heap and stack grow.
3. If you have a terminal, run `ps -o pid,ppid,comm` (Linux/macOS) or open Task Manager (Windows). Find one process and its parent. Does the parent–child relationship make sense (e.g. your terminal is the parent of a command you ran)?

A Worked Example

1. The program is the recipe, a fixed set of instructions on disk. Running it creates a process, like actually cooking the dish. You can cook the same recipe twice at once, two separate pots, so you can run the same app as two separate processes, one per window.
2. From bottom (low addresses) to top (high): **Code** (instructions), then **Data** (globals/constants), then the **Heap** growing **upward**, then free space, then the **Stack** at the top growing **downward** toward the heap.
3. For example, running `ps` from a terminal shows `ps` with its PPID pointing at the shell (`zsh` or `bash`) you typed it in, the shell is the parent that started `ps`. That makes sense: you launched it from the shell.

Course mark icon from Material Symbols by Google, Apache License 2.0.